Centro Federal de Educação Tecnológica de Minas Gerais
Professora: Poliana Corrêa

RELATÓRIO PRÁTICA 6 - LAOC
Primeira versão do processador MIPS

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

Belo Horizonte - MG
2025

1. Introdução

A prática 6 (Primeira versão do processador MIPS) , teve como foco principal implementar os três estágios iniciais de processamento de uma instrução: busca, decodificação e execução. O objetivo é reunir blocos lógicos construídos anteriormente , criando a interligação do PC, da Memória de Instruções, do Banco de Registradores e da ULA. Para o seu funcionamento, o estágio de busca utiliza o PC para obter o endereço da instrução na Memória de Instruções. Na decodificação, a instrução lida da memória alimenta o Banco de Registradores para a leitura dos dados (Read data 1 e Read data 2). No estágio de execução, os dados lidos do banco e um valor de extensão de sinal (Sign extend) são direcionados à ULA. Um multiplexador (ALUSrc) é usado para selecionar o segundo operando da ULA, que pode ser o dado do registrador (Read data 2) ou o valor estendido , permitindo a execução de diferentes tipos de instrução.

2. Desenvolvimento

Nosso projeto da Prática 6 consiste na implementação da primeira versão do processador MIPS , um componente digital que implementa os três estágios iniciais de processamento de uma instrução: busca, decodificação e execução. O módulo principal interliga os blocos fundamentais construídos anteriormente: o PC, a Memória de Instruções, o Banco de Registradores e a ULA. A arquitetura foi focada em executar um conjunto específico de duas instruções (uma do Tipo-R, add, e uma do Tipo-I, addi). Conforme especificado na prática, o projeto não implementa uma unidade de controle completa; em vez disso, os sinais de controle são "forçados" (hardcoded) com base no opcode da instrução lida.

Descrição e Decisões de Projeto dos Módulos

Contador de Programa: countPC

Descrição: O módulo countPC implementa o Program Counter (PC), que é um registrador de 32 bits responsável por armazenar o endereço da instrução que deve ser buscada. Ele possui uma entrada de reset que força o PC para zero e, a cada borda de subida do clock, atualiza seu valor para o endereço da próxima instrução.

Decisões de Projeto:

- Atualização Síncrona: A atualização do valor do PC é síncrona. Para isso, utilizamos um bloco always @(posedge CLOCK_50). Dentro deste bloco, verificamos o sinal de reset: se ativo, o PC recebe zero ({32'b0}); caso contrário, ele recebe o valor calculado pelo somador (pcout_aux).
- Incremento (PC+4): Como as instruções MIPS possuem 32 bits (4 bytes) e a memória é endereçada a byte, o PC deve ser incrementado em 4 a cada ciclo. Para realizar essa soma, decidimos instanciar um módulo somador32bits. Este somador recebe o valor atual do PC (pcin) e o valor constante 4 (representado por {29'b0, 3'b100}), gerando o endereço da próxima instrução (pcout_aux).

Memória de Instruções: memoriaInstrucao

Descrição: Este módulo implementa a memória que armazena as instruções do processador. Ele é modelado como um array de 32 posições, onde cada posição armazena uma instrução de 32 bits. A memória fornece a instrução (instruction) correspondente ao endereço de leitura (readAddress) fornecido pelo PC.

Decisões de Projeto:

- Armazenamento e Inicialização: Para armazenar as instruções, utilizamos um reg [31:0] mem [0:31]. Seguindo a exigência da prática de armazenar duas instruções específicas, usamos um bloco initial para carregar as instruções na memória. Inserimos a instrução add $s0, $t0, $zero no endereço 0 e a instrução addi $s1, $s0, 4 no endereço 1. Um laço for foi usado para zerar o restante da memória.

- Endereçamento (Conversão Byte para Palavra): O PC fornece um endereço de byte (ex: 0, 4, 8), mas o array de memória (mem) é indexado por palavra (ex: 0, 1, 2). Para fazer a conversão, a decisão foi usar apenas os bits [6:2] do readAddress para indexar a memória. Isso efetivamente descarta os dois bits menos significativos, realizando uma "divisão" por 4 e selecionando a palavra correta.

Datapath Principal: processador

Descrição: O módulo processador é a unidade central que implementa o datapath, interligando todos os componentes principais: countPC, memoriaInstrucao, bancoRegistradores, extensor_sinal, ULA32bits e os multiplexadores necessários. Ele recebe o clock e o reset e expõe as saídas principais (como o valor do PC, a instrução e o resultado da ULA) para o módulo top-level.

Decisões de Projeto:

- Lógica de Controle Forçada: A decisão mais importante deste módulo foi implementar os sinais de controle de forma "forçada", como instruído pela prática. Em vez de uma unidade de controle complexa, extraímos o opcode da instrução e usamos atribuições contínuas (assign) para definir os sinais de controle. ALUSrc é '1' se for addi (opcode 001000) , RegDest é '1' se for add (opcode 000000) , e RegWrite é sempre '1', pois ambas as instruções escrevem em registradores. O ULAop foi fixado em 3'b010 (operação de soma), pois ambas as instruções (add e addi) realizam esta operação.

- Seleção de Operando (ALUSrc): Para atender às duas instruções, um multiplexador (mux2por1_32bits) foi usado na segunda entrada da ULA. O sinal ALUSrc seleciona se a ULA receberá o dado do registrador (b, vindo do Read data 2) para a instrução add, ou o valor do extensor de sinal (sinal_estendido) para a instrução addi.

- Seleção de Registrador de Escrita (RegDest): Foi necessário um mux2por1_5bits para selecionar o registrador de destino correto. O sinal RegDest determina se o endereço de escrita (WriteReg) no banco de registradores será o campo rt (instruction[20:16], para addi) ou o campo rd (instruction[15:11], para add).

Módulo Top-Level: pratica6_top

Descrição: Este é o módulo de mais alto nível, projetado para conectar o processador aos pinos da placa FPGA. Ele mapeia as entradas físicas (CLOCK_50 e SW) para os sinais de clock, reset e seleção de display, e mapeia as saídas do

processador para os LEDs (LEDR, LEDG) e displays de 7 segmentos (HEX0 a HEX7).

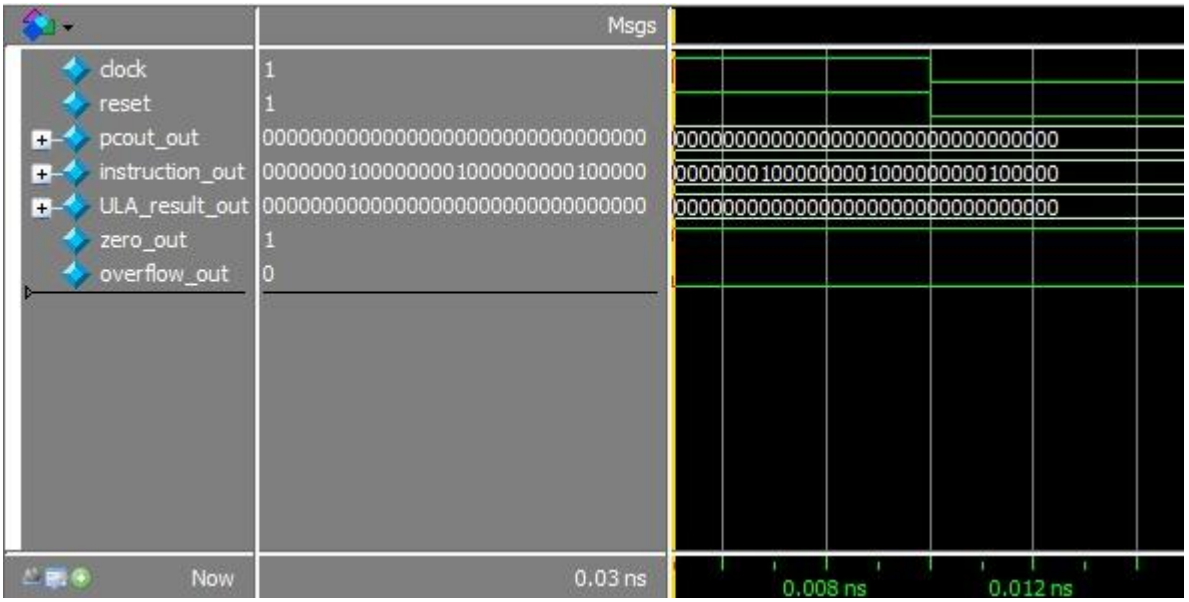
Decisões de Projeto:

- Conexão e Reset: O módulo instancia o processador. O CLOCK_50 é conectado diretamente , e o SW[17] foi escolhido como o sinal de reset global.

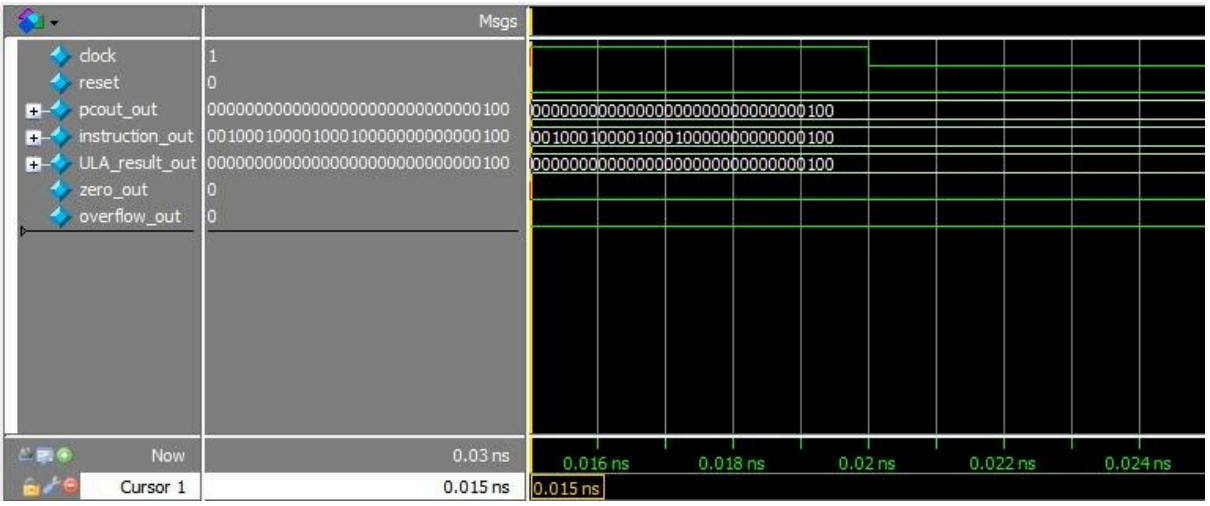
- Depuração e Visualização: Para permitir a demonstração dos estágios, implementamos uma lógica de visualização. Os switches SW[2:0] são usados como um seletor (display_sel). Um bloco always @(*) combinacional com uma estrutura case foi usado para decidir qual valor de 32 bits (pc_valor, instrucao_valor ou ULA_valor) será enviado para o registrador valor_display. Este valor é então dividido entre os 8 displays de 7 segmentos. As flags zero e overflow da ULA são conectadas diretamente aos LEDs.

3. Simulações

Inicia com reset 1 e espera o clock.



No primeiro clock, pcout inicia no endereco 0; instrucao de add é executada, faz $s0 = $t0 + $zero = 0, apresenta o resultado na ULA_result_out e flag do zero ativa



No segundo clock, o reset zera e pcout vai pro proximo endereco (PC + 4 = 0 + 4 = 4), instrucao de addi é executada, faz $s1 = $s0 + 4 = 4 e apresenta o resultado na ULA_result_out